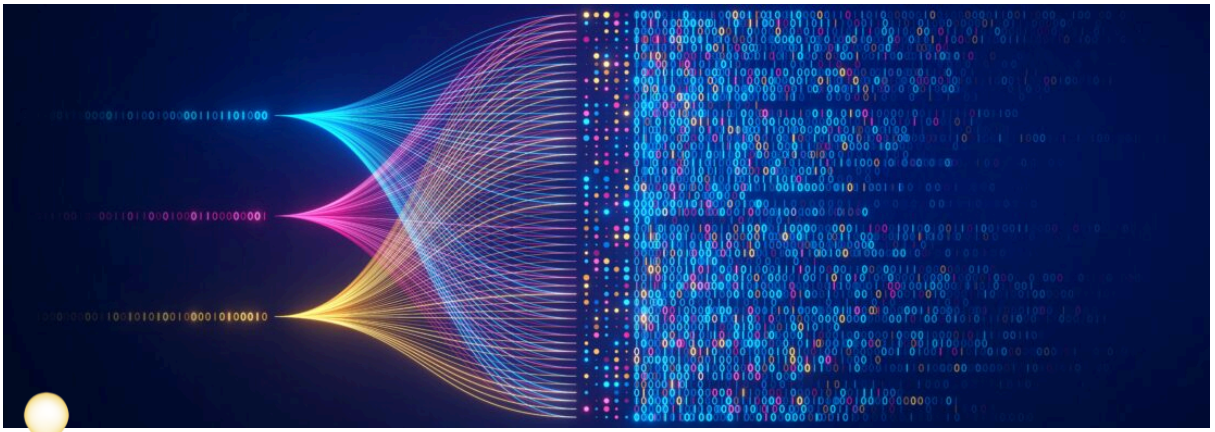


INDIVIDUAL WORK - RÚBEN PINTO - DATABASE FUNDAMENTALS - JUNE 2025

2025-06-21

Ticketing Database Documentation

▼ 1. Overview

Purpose

The Ticketing database was developed to manage key operations related to cultural event management, with a focus on tracking shows, artists, spectators, and ticket sales. It guarantees reliable linkage between performers and attendees, ensures uniqueness of seat reservations, and allows in-depth analysis of real ticket prices and participation. This structure supports clear insights into performance demand, audience behavior, and pricing strategies.

📝 **Functional Summary (based on the requirements):**

- A **show** is identified by an ID, has a title, category (e.g., classical, jazz, pop, theater, conference...), and occurs at a specific date and time.
- A **ticket** includes a reference price and links to a main artist.
- An **artist**, besides their fiscal ID (`nif`), is described by name, birth date, and type (e.g., singer, conductor, pianist...).
- If a **show is repeated**, it is treated as a new entry with a new ID, date, and time — all other attributes may repeat.
- **Spectators** are uniquely identified by email and also have name and city recorded.
- Spectators **purchase tickets** for shows, each with an associated seat and effective cost. This may match the reference price or include variations such as discounts or promotions.

🔑 **Scope**

- Manages artists, shows, spectators, and tickets
- Ensures seat uniqueness per show via composite key
- Tracks real price paid (`actual_price`) to reflect commercial scenarios
- Allows joins for business questions such as revenue analysis or artist participation

💻 **Technologies Used**

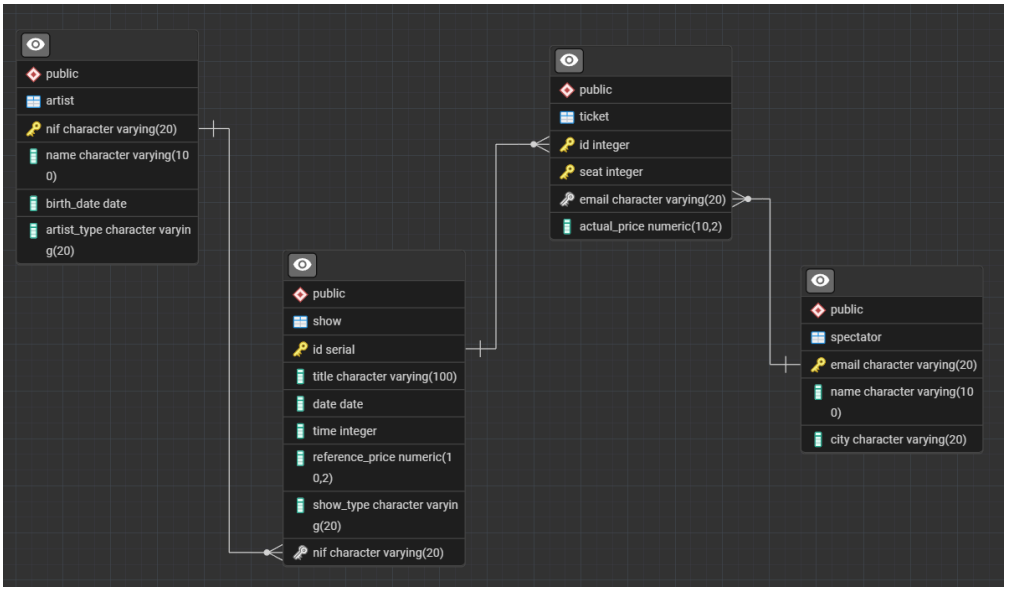
PostgreSQL via pgAdmin 4 (v7+)

▼ 2. Entity-Relationship (ER) Diagram

The ER diagram represents the relationships between the following entities:

ARTIST, **SHOW**, **SPECTATOR**, and **TICKET**.

Each entity is connected through foreign keys that enforce referential integrity. The diagram was generated using pgAdmin 4's ERD tool.



Although the schema follows a normalized design (3NF) to ensure consistency and avoid redundancy, it naturally aligns with dimensional modeling logic — treating `ticket` as the fact table and `spectator`, `show`, and `artist` as dimensions — thus resembling a **snowflake schema**. This makes the structure adaptable for analytical scenarios such as BI or reporting use cases.

▼ 3. Relationships

Artist and Show

One artist can be associated with many shows. A show has one main artist.

- Type: 1:N
- FK: `show.nif` → `artist.nif`

Show and Ticket

A show can have multiple tickets sold. **Each ticket is linked to a specific show and seat**.

- Type: 1:N
- FK: `ticket.id` → `show.id`

Spectator and Ticket

A spectator can purchase multiple tickets. **Each ticket is linked to one spectator.**

- Type: 1:N
- FK: `ticket.email` → `spectator.email`

Additional Constraint

- Composite PK on `ticket (id, seat)` ensures **seat uniqueness per show**.

4. Detailed Description of Tables

4.1 ARTIST

Purpose: Stores information about artists.

Column Name	Data Type	Description	Constraints
nif	VARCHAR(20)	Fiscal number	Primary Key
name	VARCHAR(100)	Artist's name	Not Null
birth_date	DATE	Date of birth	Nullable
artist_type	VARCHAR(20)	Type (singer, actor, etc.)	Nullable

Sample Data:

nif	name	birth_date	artist_type
1234567	Pedro Abrunhosa	02/02/1970	cantor
2345678	Carolina Deslantes	02/03/1982	cantor
1177289	Tony Carreira	1060-02-01	cantor
2999399	Rita Pereira	02/05/1982	ator
2994000	Pedro Teixeira	02/05/1980	ator

4.2 SHOW

Purpose: Stores event/show details.

Column Name	Data Type	Description	Constraints
id	SERIAL	Unique show identifier	Primary Key
title	VARCHAR(100)	Title of the show	Not Null
date	DATE	Event date	Not Null
time	INTEGER	Simplified time format for single hours (o'clock)	Not Null
reference_price	NUMERIC(10,2)	Standard price	Not Null
show_type	VARCHAR(20)	Category (concert, theater, etc.)	Nullable
nif	VARCHAR(20)	Linked artist identifier	FK → artist.nif

Sample Data:

id	title	date	time	reference_price	show_type	nif
1	Amanhecer	10/01/2021	20	20.00	musica ligeira	1234567
2	Amanhecer	11/01/2021	20	20.00	musica ligeira	1234567
3	25 anos	02/03/2021	21	40.00	musica ligeira	1177289
4	25 anos	03/03/2021	19	40.00	musica ligeira	1177289
5	Ultima hora	06/05/2021	16	15.00	teatro	2999399
6	Ultima hora	06/05/2021	21	15.00	teatro	2999399

id	title	date	time	reference_price	show_type	nif
7	Amor para a vida toda	07/08/2021	20	20.00	musica ligeira	2345678
8	Ultima hora	01/02/2022	21	15.00	teatro	2999399
9	Portugal em Festa	09/02/2022	20	10.00	teatro	2999399

▼ 4.3 SPECTATOR

Purpose: Stores spectator data.

Column Name	Data Type	Description	Constraints
email	VARCHAR(20)	Email (unique identifier)	Primary Key
name	VARCHAR(100)	Spectator name	Not Null
city	VARCHAR(20)	City of residence	Nullable

Sample Data:

email	name	city
rita@gmail.com	Rita Almeida	Braga
raquel@hotmail.com	Raquel Silva	Porto
antonio@gmail.com	Antonio Matos	Lisboa
ana@gmail.com	Ana Texeira	Porto
ricardo@hotmail.com	Ricardo Monteiro	Lisboa

▼ 4.4 TICKET

Purpose: Links a spectator to a seat in a specific show.

Column Name	Data Type	Description	Constraints
id	INTEGER	Show ID	FK → show.id
seat	INTEGER	Seat number	Part of PK
email	VARCHAR(20)	Spectator email	FK → spectator.email
actual_price	NUMERIC(10,2)	Actual price paid (after discounts)	Not Null
Composite Key	(id, seat)	Ensures seat uniqueness per show	Primary Key

Sample Data:

id	seat	email	actual_price
3	2	rita@gmail.com	60.00
3	10	raquel@hotmail.com	40.00
4	12	raquel@hotmail.com	35.00
1	56	antonio@gmail.com	25.00
1	57	ana@gmail.com	25.00
5	30	ricardo@hotmail.com	20.00

▼ 5. SQL Scripts

Before performing queries or extracting insights, the first step in any database-driven project is defining the **physical data model** — that is, the concrete structure where data will be stored. It's creation is suported on the ERD on section 2.

The SQL code below creates the necessary schema for the **Ticketing** system, including four core tables: `artist`, `show`, `spectator`, and `ticket`. It begins by safely removing any existing versions of the tables (to allow for reruns during development), and then defines each table's fields, data types, primary keys, and foreign key relationships to ensure referential integrity.

This structure ensures that every ticket sold is properly linked to both a show and a spectator, and that each show is associated with a registered artist.

For this project, data is populated via `.csv` files imported through **pgAdmin**, but in real-world data pipelines — especially in data engineering contexts — databases are often populated via **ETL tools** like **Apache NiFi**, **Airflow**, or custom scripts in **Python** or **Scala**. These tools extract data from sources such as APIs, flat files, or streaming platforms like **Kafka**, and load it efficiently into PostgreSQL, cloud warehouses (e.g., Snowflake, BigQuery), or distributed systems like **Apache Hive**. Regardless of the ingestion method, designing a clean, normalized schema — as shown below — remains a fundamental best practice for long-term maintainability and accurate analytics.

▼ 5.1 Table Creation & Record Insert Script

```
-- for reruns
DROP TABLE IF EXISTS public.ticket;
DROP TABLE IF EXISTS public.spectator;
DROP TABLE IF EXISTS public.show;
DROP TABLE IF EXISTS public.artist;


-- Table: artist
CREATE TABLE public.artist(
    nif character varying (20) PRIMARY KEY,  -- since it won't be used for for mathematical operations, deci
    name character varying(100) NOT NULL,
    birth_date date,
    artist_type character varying (20)
);

-- Table: show
CREATE TABLE public.show(
    id serial PRIMARY KEY,  /* didnt use numeric since we dont need
    decimal-like */
    title character varying(100) NOT NULL,
    date date NOT NULL,
    time integer NOT NULL,  /* integer to allow import,
    assumed registering at o'clock hours */
    reference_price numeric (10,2) NOT NULL,
    show_type character varying (20),
    nif character varying (20) REFERENCES public.artist(nif)

    /* or nif serial and \n FOREIGN KEY (nif) REFERENCES public.artist(nif) */
    /*
        → This column nif will store values that must exist in public.artist(nif).

        So PostgreSQL:

        - Enforces referential integrity

        - Will block inserts/updates,
                if the nif value doesn't exist in public.artist(nif)
    */
);

-- Table: spectator
CREATE TABLE public.spectator(
    email varchar(20) PRIMARY KEY,
    name character varying (100) NOT NULL,
```

```
    city character varying (20)
);

-- Table: ticket
CREATE TABLE public.ticket(
    id integer,
    FOREIGN KEY (id)  REFERENCES public.show(id),
    seat integer NOT NULL,
    email varchar(20),
    FOREIGN KEY (email)  REFERENCES public.spectator(email),
    /*it shouldn't be and email on PK conception as good pratice...
    since people can change it and then we lose track of the record for example*/
    actual_price numeric (10,2) NOT NULL,
    PRIMARY KEY (id, seat)
    /* to ensure the same seat in that show isn't booked more than once;
    they are FK, so nothin is stoping the two attibutes to repeat... */
    -- uniqueness of seats per show by using a composite key concept
);
```

▼ 6. Queries and Reports

This section presents a list of queries that can be executed on the database to gain insights for operational tasks or to support business decision-making.

Introduction to Query Analysis

With the database schema fully implemented and populated, the next step involves exploring and validating the stored data through SQL queries. Although the dataset used is intentionally small for academic purposes, the structure allows simulation of real-world operations such as retrieving event schedules, tracking ticket sales, and analyzing participant profiles.

The following queries are designed to answer concrete business questions based on the project's requirements — from listing shows and identifying audience profiles, to calculating revenue per event or artist. They serve as proof of concept for the relational model previously built, leveraging **joins**, **aggregations**, **filtering**, and in some cases **subqueries** or **CTEs** (Common Table Expressions) to ensure clarity and modular logic.

While this project uses static data inserted via CSVs, in a production setting, these types of queries would form the backbone of **reporting dashboards**, **monitoring scripts**, or be embedded in **BI tools** like Power BI, Tableau, or Metabase — especially when dealing with much larger volumes of data.

The queries are presented in a consistent format: each one begins with the question, followed by the SQL code used to answer it, and ends with the result generated from the current dataset.

▼ P1) What are the titles, dates, and times of the shows registered in the database?

```
SELECT *
FROM   public.SHOW
LIMIT  5;

SELECT title,
       date,
       time
FROM   public.SHOW
```

	title character varying (100)	date date	time integer
1	Amanhecer	2021-01-10	20
2	Amanhecer	2021-01-11	20
3	25 anos	2021-03-02	21
4	25 anos	2021-03-03	19
5	Ultima hora	2021-05-06	16
6	Ultima hora	2021-05-06	21
7	Amor para a vida toda	2021-08-07	20
8	Ultima hora	2022-02-01	21
9	Portugal em Festa	2022-02-09	20

▼ P2) What are the categories and titles of the shows registered in the database?

```
SELECT DISTINCT show_type,
      title
FROM   PUBLIC.show
```

	show_type character varying (20)	title character varying (100)
1	teatro	Ultima hora
2	musica ligeira	Amanhecer
3	teatro	Portugal em Festa
4	musica ligeira	Amor para a vida toda
5	musica ligeira	25 anos

▼ P3) What are the names and birth dates of the registered singers? Sort the result by birth date in ascending order.

```
SELECT name,
      birth_date
FROM   public.artist
ORDER  BY birth_date ASC
```

	name character varying (100)	birth_date date
1	Tony Carreira	1060-02-01
2	Pedro Abrunhosa	1970-02-02
3	Pedro Teixeira	1980-05-02
4	Carolina Deslantes	1982-03-02
5	Rita Pereira	1982-05-02

▼ P4) What are the emails and names of the spectators from Porto? Sort the result by email.

```
SELECT email,
      name
FROM   public.spectator
WHERE  1 = 1
      AND city = 'Porto'
ORDER  BY email
```

	email [PK] character varying (20)	name character varying (100)
1	ana@gmail.com	Ana Texeira
2	raquel@hotmail.com	Raquel Silva

▼ P5) For each artist, list their name along with the date and title of the shows in which they were the main artist. Sort the result by artist name (ascending) and show date (descending).

```
SELECT a.NAME,
       s.date,
       s.title
FROM   PUBLIC.artist a
       LEFT JOIN PUBLIC.show s
              ON s.nif = a.nif
ORDER  BY a.NAME ASC,
          s.date DESC
```

	name character varying (100)	date date	title character varying (100)
1	Carolina Deslantes	2021-08-07	Amor para a vida toda
2	Pedro Abrunhosa	2021-01-11	Amanhecer
3	Pedro Abrunhosa	2021-01-10	Amanhecer
4	Pedro Teixeira	[null]	[null]
5	Rita Pereira	2022-02-09	Portugal em Festa
6	Rita Pereira	2022-02-01	Ultima hora
7	Rita Pereira	2021-05-06	Ultima hora
8	Rita Pereira	2021-05-06	Ultima hora
9	Tony Carreira	2021-03-03	25 anos
10	Tony Carreira	2021-03-02	25 anos

▼ P6) For each spectator, list their name along with the seat and the cost of the tickets they purchased. Sort the result by spectator name (ascending) and ticket cost (descending).

```
SELECT sp.NAME,
       t.seat,
       t.actual_price
FROM   PUBLIC.spectator sp
       LEFT JOIN PUBLIC.ticket t
              ON t.email = sp.email
ORDER  BY sp.NAME ASC,
          t.actual_price DESC;
```

	name character varying (100)	seat integer	actual_price numeric (10,2)
1	Ana Texeira	57	25.00
2	Antonio Matos	56	25.00
3	Raquel Silva	10	40.00
4	Raquel Silva	12	35.00
5	Ricardo Monteiro	30	20.00
6	Rita Almeida	2	60.00

▼ P7) List, without duplicates, the names of the spectators who bought tickets for shows, along with the name of the artist.

```
SELECT DISTINCT sp.NAME AS spectator_name,
                a.NAME  AS artist_name
FROM   PUBLIC.spectator sp
       INNER JOIN PUBLIC.ticket t
               ON t.email = sp.email
       LEFT JOIN PUBLIC.show s
              ON s.id = t.id
       LEFT JOIN PUBLIC.artist a
              ON a.nif = s.nif
```

	spectator_name character varying (100)	artist_name character varying (100)
1	Ana Texeira	Pedro Abrunhosa
2	Antonio Matos	Pedro Abrunhosa
3	Rita Almeida	Tony Carreira
4	Ricardo Monteiro	Rita Pereira
5	Raquel Silva	Tony Carreira

▼ P8) List the names of all people — both spectators and artists — as long as the spectators are from Porto and the artists are actors.

```
SELECT NAME
FROM   PUBLIC.spectator
WHERE  1 = 1
       AND city = 'Porto'
UNION
SELECT NAME
FROM   PUBLIC.artist
WHERE  1 = 1
       AND artist_type = 'ator'
```

	name character varying (100)
1	Ana Texeira
2	Pedro Teixeira
3	Raquel Silva
4	Rita Pereira

▼ P9) List the names of all people who are not from Porto and who are not artist names.

```
SELECT NAME
FROM   PUBLIC.spectator
WHERE  1 = 1
       AND city != 'Porto'
       AND NAME NOT IN (SELECT NAME
                        FROM   PUBLIC.artist)
```

	name character varying (100)
1	Rita Almeida
2	Antonio Matos
3	Ricardo Monteiro

▼ P10) What is the total revenue per show? Show the show ID, title, and total ticket value.

```
SELECT s.id,
       s.title,
       COALESCE(Sum(t.actual_price), 0) AS total_actual_revenue
FROM   PUBLIC.show s
       LEFT JOIN PUBLIC.ticket t
              ON t.id = s.id
WHERE  1 = 1
       AND s.date < CURRENT_DATE
GROUP  BY s.id,
          s.title
ORDER  BY s.id;
```

	id [PK] integer	title character varying (100)	total_actual_revenue numeric
1	1	Amanhecer	50.00
2	2	Amanhecer	0
3	3	25 anos	100.00
4	4	25 anos	35.00
5	5	Ultima hora	20.00
6	6	Ultima hora	0
7	7	Amor para a vida toda	0
8	8	Ultima hora	0
9	9	Portugal em Festa	0

▼ P11) What is the total revenue per show category? Show the category and the total ticket value.

```
SELECT s.show_type,
       COALESCE(Sum(t.actual_price), 0) AS total_actual_revenue
FROM   PUBLIC.show s
       LEFT JOIN PUBLIC.ticket t
              ON t.id = s.id
WHERE  1 = 1
       AND s.date < CURRENT_DATE
GROUP  BY 1
ORDER  BY 1;
```

	show_type character varying (20)	total_actual_revenue numeric
1	musica ligeira	185.00
2	teatro	20.00

▼ P12) Which spectator(s) (email) attended all shows by Tony Carreira?

```
-- subquery usage + having count
SELECT sp.email,
       sp.NAME,
       Count(*) AS TC_attendance_number
FROM   PUBLIC.spectator sp
       INNER JOIN PUBLIC.ticket t
              ON t.email = sp.email
       INNER JOIN PUBLIC.show s
              ON s.id = t.id
       INNER JOIN PUBLIC.artist a
              ON a.nif = s.nif
WHERE  1 = 1
```

```
      AND a.NAME = 'Tony Carreira'
GROUP  BY 1,
          2
HAVING Count(*) = (SELECT Count(*) AS TC_shows_number
                   FROM   PUBLIC.show s
                          INNER JOIN PUBLIC.artist a
                                  ON a.nif = s.nif
                   WHERE  1 = 1
                          AND a.NAME = 'Tony Carreira')
```

	email [PK] character varying (20)	name character varying (100)	tc_attendance_number bigint
1	raquel@hotmail.com	Raquel Silva	2

```
-- with CTE expression

WITH tc_shows_count
     AS (SELECT Count(*) AS TC_shows_number
         FROM   PUBLIC.show s
                INNER JOIN PUBLIC.artist a
                        ON a.nif = s.nif
         WHERE  1 = 1
                AND a.NAME = 'Tony Carreira'),
     tc_attendance_track
     AS (SELECT sp.email,
                sp.NAME,
                Count(*) AS TC_attendance_number
         FROM   PUBLIC.spectator sp
                INNER JOIN PUBLIC.ticket t
                        ON t.email = sp.email
                INNER JOIN PUBLIC.show s
                        ON s.id = t.id
                INNER JOIN PUBLIC.artist a
                        ON a.nif = s.nif
         WHERE  1 = 1
                AND a.NAME = 'Tony Carreira'
         GROUP  BY 1,
                   2)
SELECT ta.email,
       ta.NAME
FROM   tc_attendance_track ta
       INNER JOIN tc_shows_count tc
               ON tc.tc_shows_number = ta.tc_attendance_number
```

7. Change Log

Date	Description	Author
2025-06-17	Initial schema design and ERD	Rúben Pinto
2025-06-18	CSV data loading and normalization checks	Rúben Pinto
2025-06-19	SQL queries implemented (P1–P12)	Rúben Pinto
2025-06-20	Document Revision	Rúben Pinto
2025-06-21	Send Final Project	Rúben Pinto

8. Future Improvements

While this project is built with a simplified dataset, its structure allows further expansion and practice in more advanced areas of PostgreSQL and data engineering.

- 📌 Add additional tables (e.g., `venue`, `payment`, `promotion`)
- 📌 Create reusable `VIEW`s and `MATERIALIZED VIEW`s for reporting
- 📌 Implement `TRIGGER`s to enforce business rules or log changes
- 📌 Explore use of `JSONB` columns for unstructured metadata (e.g., artist bios)
- 📌 Simulate data ingestion with Python scripts or mock APIs
- 📌 Connect the database to Power BI or Metabase for live dashboards
- 📌 Deploy a version of the database in a containerized environment (e.g., Docker)

---

9. Conclusion

This project served as a practical foundation to apply relational database modeling, data normalization, and SQL querying using PostgreSQL. From schema definition to business-oriented queries, it reflects good practices in database design and documentation.

Although based on a limited dataset, the logic, constraints, and use of SQL features like `JOINs`, `HAVING`, `CTEs`, and `COALESCE` were applied with real-world considerations in mind. It provides a scalable base that can evolve into a broader project — whether for portfolio development, integration with BI tools, or as a sandbox for further practice in data engineering and automation with tools like Airflow or Python.